

FINAL PROJECT REPORT

Fake News Detection using NLP and Machine Learning

Machine Learning System for Fake News Detection in Online Media

Python | NLP | TF-IDF | Scikit-learn | Text Classification | Dashboard Reporting

Student Name	Chiak Ndi
Module / Training	Machine Learning / Data Science
Mentor / Professor	Manisha
Dataset	Fake and Real News Dataset (Kaggle, provided as two CSV files)
Submission	Final report with methodology, dashboard explanation, model evaluation, and conclusion

Date: July 2026

Executive Summary

This project develops a machine learning text-classification system that automatically classifies online news articles as Fake News or Real News. The project addresses the real-world challenge of misinformation on digital platforms, where users may struggle to distinguish legitimate journalism from fabricated content. The work follows a complete data science workflow: data loading, CSV repair, data cleaning, exploratory data analysis, dashboard visualization, text preprocessing, TF-IDF feature extraction, model training, model evaluation, interpretation, and final conclusion.

The two professor-provided CSV files were labelled and combined into one dataset. The fake-news dataset was assigned label 0 and the real-news dataset was assigned label 1. After cleaning and duplicate handling, the notebook notes report 3,574 final rows, no missing values, 2,939 unique titles, and 3,563 unique original texts. The cleaned title and article body were merged into a single text field and transformed using TF-IDF vectorization. Three supervised learning models were trained and compared: Logistic Regression-style SGDClassifier, Multinomial Naive Bayes, and Linear SVM.

The best-performing model in the final notebook notes was the Linear SVM model, with an F1-score of approximately 98.83%. The confusion matrix from the final model run showed very few incorrect predictions, demonstrating that the system learned strong linguistic patterns that distinguish fake and real news articles.

1. Problem Statement

The rapid spread of misinformation on online media platforms is a major social and technological challenge. False information can influence public opinion, political decisions, health choices, and trust in journalism. Because online users are exposed to a very large volume of news content, manual verification is difficult and slow. This project therefore builds an automated machine learning system that can classify news articles as Fake or Real based on their textual content.

The final objective is not only to produce a high-performing classification model, but also to explain the process clearly through data analysis, model evaluation, and dashboard visuals that make the results understandable to both technical and non-technical audiences.

2. Project Objectives

- Perform Exploratory Data Analysis (EDA) to understand the structure and linguistic differences between fake and real news articles.
- Clean and preprocess raw news text by removing unnecessary characters, links, symbols, numbers, duplicates, and extra spaces.
- Convert text into numerical machine learning features using TF-IDF vectorization.
- Train and compare multiple classification models for fake news detection.
- Evaluate model performance using accuracy, precision, recall, F1-score, ROC/AUC, and confusion matrices.

- Identify important terms and patterns that influence fake and real news classification.
- Build a reusable machine learning pipeline that can classify unseen news articles.
- Create dashboard-style visuals that explain the dataset, model performance, confusion matrix, and important terms.

3. Dataset Description and Labelling

The project used two CSV datasets supplied for the assignment. One file contained fake news articles and the other contained real news articles. Since the machine learning model needs a target variable, each dataset was labelled before being combined.

Dataset file	Meaning	Assigned label
csv data set 1.csv	Fake news articles	0 = Fake
csv data set 2.csv	Real news articles	1 = Real

The extracted fields from the CSV files were title, text, subject, date, label, and label_name. The title and text fields were later combined because the headline and article body together provide stronger context for classification.

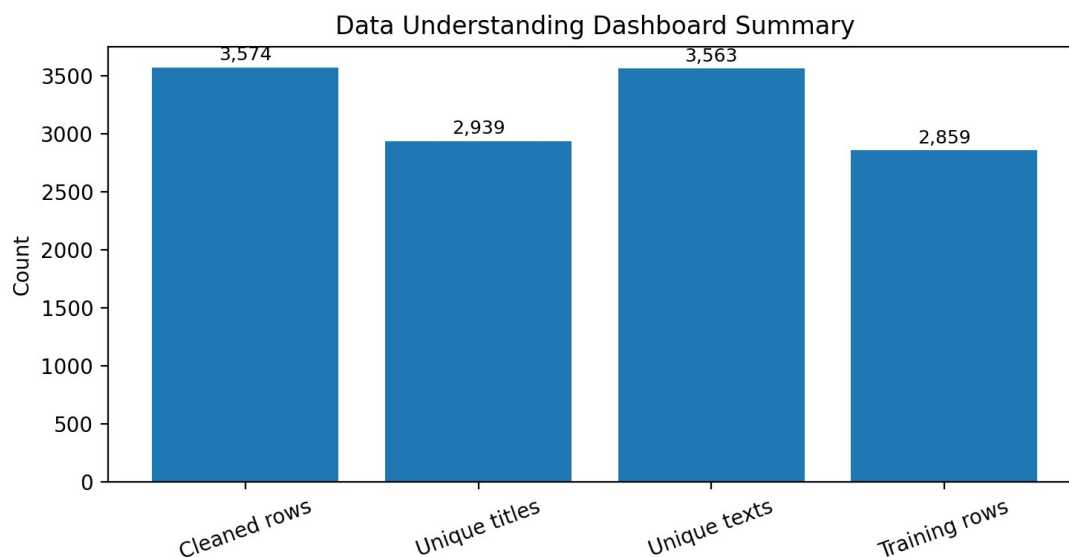


Figure 1. Data understanding dashboard summary from the notebook notes.

4. Tools and Libraries Used

The project was implemented in Python using Google Colab / Jupyter Notebook. The main libraries and their purposes are summarized below.

Library / Tool	Purpose in this project
pandas	Load, combine, clean, inspect, and manage tabular datasets.
numpy	Support numerical operations and array calculations.
csv	Read semi-structured CSV rows and repair rows with

	semicolon delimiters.
re	Clean text using regular expressions, such as removing URLs, symbols, and numbers.
matplotlib / seaborn	Create EDA graphs, dashboard visuals, histograms, boxplots, and comparison charts.
scikit-learn	Split data, build TF-IDF features, train models, and evaluate performance.
TfidfVectorizer	Convert cleaned text into numerical features for machine learning.
SGDClassifier, MultinomialNB, Linear SVM	Train supervised text classification models.
classification_report, confusion_matrix, ROC/AUC	Evaluate the quality of the trained models.

5. Methodology

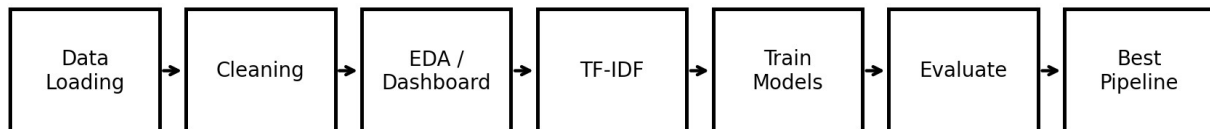


Figure 2. End-to-end machine learning workflow used in the project.

5.1 Data Loading and CSV Repair

The CSV files were read using a custom function because the data rows were separated with semicolons and some rows required careful handling. The function extracted the headline, article body, subject, and date, then added the numeric class label and readable label name. This avoided row-index errors and made the data ready for analysis.

```

def read_news_file(path, label_name, label):
    rows = []
    with open(path, "r", encoding="utf-8", errors="ignore") as file:
        reader = csv.reader(file, delimiter=";")
        for row in reader:
            if len(row) >= 4:
                title = row[0]
                text = " ".join(row[1:-2])
                subject = row[-2]
                date = row[-1]
                rows.append({
                    "title": title,
                    "text": text,
                    "subject": subject,
                    "date": date,
                    "label": label,
                    "label_name": label_name
                })
    return rows

```

5.2 Data Cleaning

The cleaning stage removed missing or empty article texts, stripped unnecessary spaces, and dropped duplicate articles based on title and text. This was important because duplicated articles can make the model look more accurate than it really is.

Cleaning action	Reason
Strip spaces from title, text, subject, and date	Standardizes the dataset and avoids invisible formatting problems.
Remove empty text rows	A model cannot learn from articles without text.
Drop duplicates on title and text	Reduces bias and prevents repeated articles from inflating performance.
Check missing values	Confirms that the final dataset is complete before modelling.

5.3 Text Preprocessing

The title and article body were combined into one field called `combined_text`. A text-cleaning function then converted the text to lowercase, removed web links, removed non-letter characters, removed extra spaces, and created a `word_count` variable for article-length analysis.

Step	Explanation
Lowercase conversion	Makes “News”, “news”, and “NEWS” the same word.
Remove links	URLs are usually not useful for linguistic classification.
Remove numbers and symbols	Keeps the model focused on words and reduces noise.
Remove extra spaces	Creates consistent cleaned text.
Word count creation	Supports dashboard analysis of article length differences.

5.4 Exploratory Data Analysis and Dashboard

EDA was used to understand the dataset before modelling. The dashboard visuals included class distribution, subject distribution, word count distribution, article length comparison between fake and real news, model performance comparison, confusion matrices, and important words associated with each class.

Dashboard visual	Question answered	Interpretation
Fake vs Real class distribution	Is the dataset balanced?	Shows whether one class dominates the other, which can affect model learning.
Top 10 subjects	Which news categories appear most?	Reveals the dominant article topics in the dataset.
Word count histogram	How long are the articles?	Shows the distribution of article lengths and possible outliers.
Fake vs Real word-count boxplot	Do fake and real articles differ in length?	Compares central tendency and spread by class.
Model performance comparison	Which model performs best?	Compares accuracy, precision, recall, and F1-score across models.
Confusion matrix	What errors does the best model make?	Shows false fake and false real predictions.
Important terms	What words influence predictions?	Improves interpretability by

		showing influential fake/real terms.
--	--	--------------------------------------

6. Feature Engineering: TF-IDF Vectorization

Machine learning models cannot directly understand raw text, so the cleaned text was converted into numerical features using TF-IDF. TF-IDF stands for Term Frequency-Inverse Document Frequency. It gives higher importance to words that are frequent in a specific article but not common across all articles. This makes it useful for text classification because it reduces the weight of very common words and highlights more informative terms.

In the notebook notes, the TF-IDF training matrix had the shape (2,859, 41,093). This means 2,859 training articles were represented using 41,093 unique text features after vectorization. The training and testing split used an 80/20 split with stratification, helping preserve the fake/real class balance in both sets.

7. Model Training

Three supervised classification models were trained and compared. The models were placed inside pipelines so that TF-IDF vectorization and model fitting could be applied together in a reusable way.

Model	Why it was used	Expected strength
Logistic Regression-style SGDClassifier	A strong linear model for text classification using log-loss.	Good baseline and interpretable coefficients.
Multinomial Naive Bayes	Common and fast model for word-count and TF-IDF text classification.	Simple, efficient, and suitable for NLP baselines.
Linear SVM	A powerful linear classifier for high-dimensional sparse text data.	Often performs very well on TF-IDF text features.

8. Model Evaluation

The models were evaluated using standard classification metrics: accuracy, precision, recall, F1-score, confusion matrix, and ROC/AUC. These metrics give a more complete view than accuracy alone, especially because fake news detection requires both fake and real articles to be correctly identified.

Metric	Meaning in this project
Accuracy	Overall percentage of correctly classified articles.
Precision	Among articles predicted as a class, the percentage that truly belong to that class.
Recall	Among true articles of a class, the percentage the model correctly finds.
F1-score	Balanced measure combining precision and recall.
Confusion matrix	Shows correct and incorrect predictions by class.
ROC/AUC	Shows how well the model separates fake and real news across thresholds.

9. Results and Findings

The final notebook notes identify Linear SVM as the best-performing model, with an F1-score of approximately 98.83%. This indicates that the model achieved a very strong balance between precision and recall for fake and real news classification.

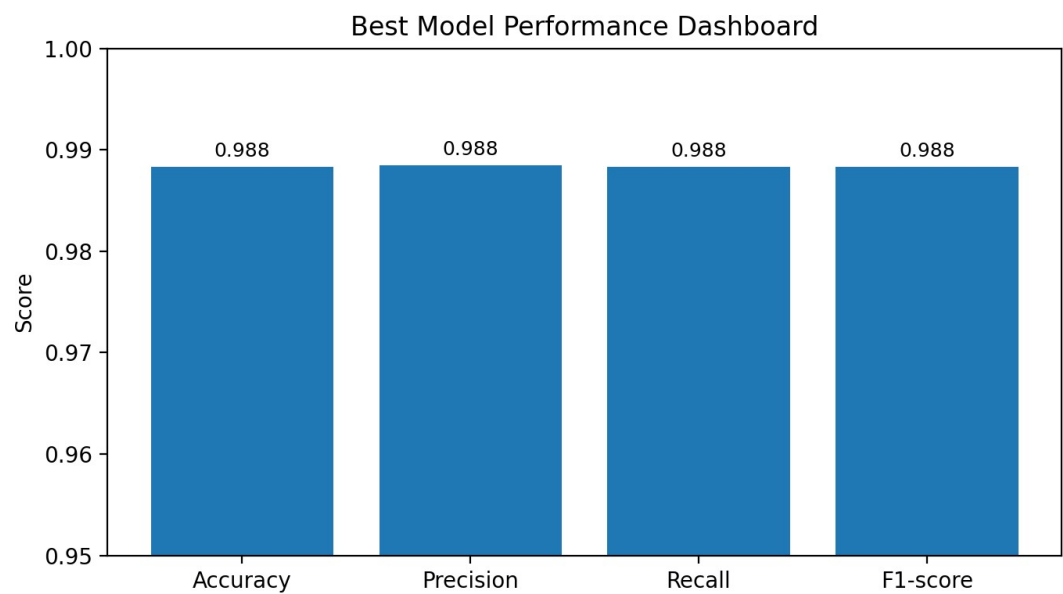


Figure 3. Best model performance dashboard based on the reported final confusion matrix.

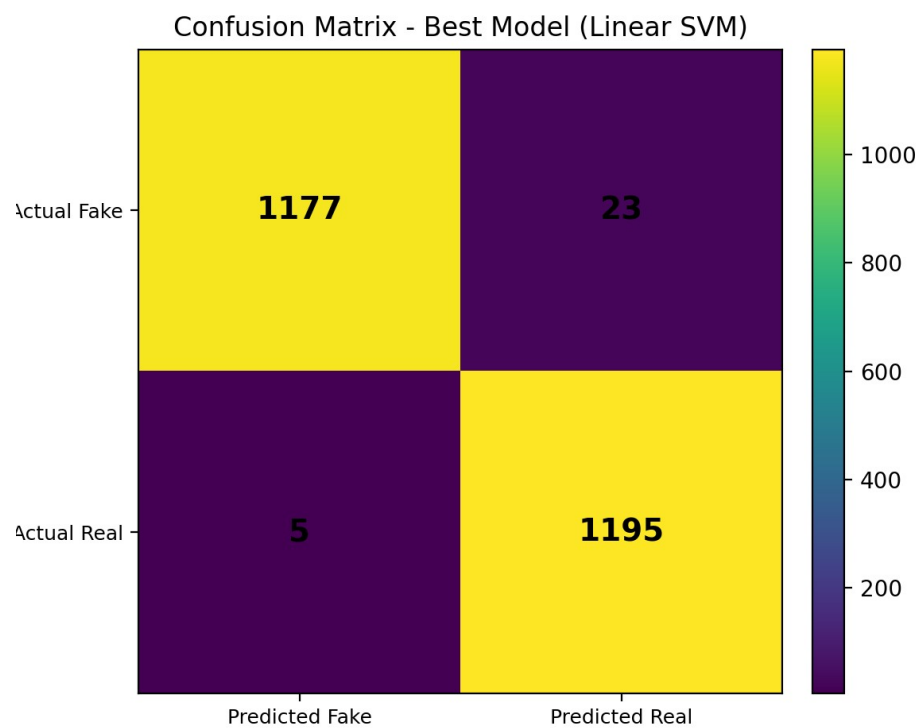


Figure 4. Confusion matrix for the best model from the final notebook notes.

	Predicted Fake	Predicted Real
--	----------------	----------------

Actual Fake	1,177	23
Actual Real	5	1,195

Interpretation: The model correctly classified most articles. The reported final confusion matrix shows 28 wrong predictions out of 2,400 articles in that final evaluation run. The errors consisted of 23 fake articles predicted as real and 5 real articles predicted as fake. This suggests that the final model was especially strong at identifying real news while also maintaining high fake-news detection performance.

10. Model Interpretation: Important Fake and Real Terms

To make the model more explainable, an interpretation pipeline was trained using TF-IDF and a logistic regression-style linear classifier. The feature coefficients were inspected. Positive coefficients were interpreted as terms more associated with Real news, while negative coefficients were interpreted as terms more associated with Fake news.

This interpretation step is important because fake news detection should not be treated only as a black-box prediction task. By reviewing influential terms, the analyst can understand what patterns the model uses and check whether the model is learning meaningful language signals rather than irrelevant noise.

11. Reusable Prediction Pipeline

The final system is reusable because the TF-IDF vectorizer and classifier are combined in a pipeline. A new unseen news article can be passed through the same cleaning and prediction process to obtain a Fake or Real prediction.

1. Input unseen news title and article text.
2. Combine title and article body into one text field.
3. Apply the same cleaning function used during training.
4. Transform the cleaned text using the trained TF-IDF vectorizer.
5. Predict the class using the trained best model.
6. Return the result as Fake News or Real News.

12. Discussion

The results show that NLP and classical machine learning methods can perform very well on fake news detection when the text is cleaned properly and represented with strong features such as TF-IDF. Linear models are suitable for this problem because TF-IDF creates a high-dimensional sparse feature matrix, and models such as Linear SVM are designed to handle this type of data efficiently.

The dashboard component is also important. It helps the professor or project reviewer quickly understand the dataset, article-length patterns, class distribution, subject distribution, model comparison, and prediction errors. Instead of presenting only code, the dashboard visuals make the work easier to evaluate and explain.

13. Limitations and Future Improvements

- **Dataset source limitation:** The model was trained on the provided Kaggle-style fake and real news dataset. Performance may change on newer articles, different languages, or different media sources.
- **Possible topic bias:** If fake and real datasets contain different subject distributions, the model may learn topic patterns instead of only truthfulness signals.
- **Classical NLP limitation:** TF-IDF captures word importance but does not fully capture deep context, sarcasm, or complex meaning.
- **Need for real-world validation:** Before deployment, the model should be tested on fresh unseen news from multiple sources.
- **Future improvement:** Use lemmatization, cross-validation, hyperparameter tuning, explainability tools, and transformer-based models such as BERT for comparison.

14. Final Conclusion

This project successfully built a machine learning system for fake news detection using Natural Language Processing. The two datasets were loaded, repaired, labelled, cleaned, combined, and explored. Text preprocessing was applied to remove noise and prepare the data for modelling. TF-IDF vectorization transformed the cleaned text into numerical features suitable for machine learning.

Three models were trained and compared: Logistic Regression-style SGDClassifier, Multinomial Naive Bayes, and Linear SVM. Based on the final notebook notes, Linear SVM achieved the best overall performance, with an F1-score of approximately 98.83%. The final confusion matrix showed only a small number of wrong predictions, confirming that the model learned useful linguistic patterns for distinguishing fake and real news articles.

The project demonstrates practical skills in Python, data cleaning, exploratory data analysis, NLP, TF-IDF vectorization, supervised machine learning, model evaluation, dashboard reporting, and project communication. It also addresses a meaningful real-world problem: improving digital media credibility and supporting the fight against misinformation.

15. References

- Professor project brief: Machine Learning System for Fake News Detection in Online Media, Manisha.
- Kaggle Fake and Real News Dataset, as specified in the project brief.
- Scikit-learn documentation for TF-IDF vectorization, pipelines, classifiers, and model evaluation metrics.
- Project notebook notes and generated outputs supplied for the final report preparation.

Appendix A: Key Code Structure Used in the Notebook

1. Import libraries: pandas, numpy, csv, re, matplotlib, seaborn, and scikit-learn.
2. Load fake and real CSV files using `read_news_file()`.
3. Assign labels: 0 for Fake and 1 for Real.
4. Combine both datasets into one dataframe.
5. Clean title, text, subject, and date columns.
6. Remove empty texts and duplicate title/text combinations.
7. Create `combined_text` and `clean_text` columns.
8. Create `word_count` for dashboard analysis.
9. Split data into train and test sets using stratified 80/20 split.
10. Apply TF-IDF vectorization.
11. Train Logistic Regression-style `SGDClassifier`, Naive Bayes, and Linear SVM.
12. Evaluate with classification report, confusion matrix, ROC/AUC, and model comparison chart.
13. Select the best model based on F1-score.
14. Interpret important fake and real terms.
15. Present dashboard visuals and final conclusion.